



复习课（二）

（自底向上文法解析）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，一次归约实质上是一个推导的反向操作
 - 核心操作：移进（shift）与 归约（reduce）



自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作



自底向上的语法分析

- 为一个输入串构造语法分析树的过程
- 从叶子（输入串中的终结符号，将位于分析树的底端）开始，向上到达根结点
- 重要的自底向上语法分析的通用框架
 - 移入-归约
- LR：最大的可以构造出移入-归约语法分析器的语法类



自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，一次归约实质上是一个推导的反向操作（重要！）



自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作
- 自底向上分析的整条归约序列，等价于某个“最右推导（rightmost derivation）”的逆序
如果：

$$S = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n = w$$

是一条最右推导，那么自底向上的分析从 w 出发，按

$$\gamma_n \xrightarrow{\text{reduce}} \gamma_{n-1} \xrightarrow{\text{reduce}} \dots \xrightarrow{\text{reduce}} \gamma_0 = S$$

逐步把右部还原为左部，直到得到开始符号 S



自底向上的语法分析



- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，**一次归约实质上是一个推导的反向操作（重要！）**
 - 核心操作：**移进（shift）** 与 **归约（reduce）**
 - 主要解决问题：
 - 什么时候应该移进
 - 什么时候应该归约



自底向上的语法分析



- 问题来了：**冲突**
- 为什么会有冲突：
 - 移入-归约技术并不能处理所有上下文无关文法



自底向上的语法分析



- 规约-规约冲突
- 示例：
 - 输入：id and id or id
 - 语义上我们希望：
 - and 的优先级 高于 or
 - 即应该解析为 (id and id) or id
 - 但是在这个文法下，LR 分析器会遇到规约-规约冲突：
 - 在读入 id and id 之后，分析栈已经能规约为一个 E
 - 但同时，前两个 id 也能直接规约为 $E \rightarrow id$ （多种规约选择）
 - 在下一个符号 or 到来时，分析器会发现：
 - 可以规约 $E \rightarrow E \text{ and } E$
 - 也可以规约 $E \rightarrow id$ （针对最后那个 id）
 - 两个不同的 reduce 操作同时出现 $\Rightarrow$ 规约-规约冲突

$E \rightarrow E \text{ and } E$
 $E \rightarrow E \text{ or } E$
 $E \rightarrow id$



自底向上的语法分析



- 规约-规约冲突
- 示例：

(1)	stmt	$\rightarrow$	id (parameter_list)
(2)	stmt	$\rightarrow$	expr := expr
(3)	parameter_list	$\rightarrow$	parameter_list , parameter
(4)	parameter_list	$\rightarrow$	parameter
(5)	parameter	$\rightarrow$	id
(6)	expr	$\rightarrow$	id (expr_list)
(7)	expr	$\rightarrow$	id
(8)	expr_list	$\rightarrow$	expr_list , expr
(9)	expr_list	$\rightarrow$	expr



自底向上的语法分析



- 问题来了：**冲突**
- 为什么会有冲突：
 - 文法允许多种推导方式（即 二义性文法）
 - 语法分析器无法仅凭当前状态和下一个输入符号决定：
 - 现在就归约
 - 还是继续移进
 - 或者说，应该选择哪条产生式进行规约
- 两类冲突：
 - 移进-规约冲突：分析器无法决定“移进”还是“规约”
 - 规约-规约冲突：分析器无法决定“用哪条规则规约”



自底向上的语法分析



- 从输入串出发构造一个反向最右推导序列（归约）
- 每一步是对**句柄**进行归约 -- 寻找句柄
- 一种移进-归约的分析框架
 - 一个栈存放文法符号，一个输入缓冲区
 - 移进 or 归约
 - 总是能够在栈的顶端找到句柄
 - 通过证明是可行的
- 有时候会有冲突，是移进还是归约？怎么归约？



自底向上的语法分析



- LR语法分析技术
- LR(k)的语法分析概念
 - L表示最左扫描, R表示反向构造出最右推导
 - k表示最多向前看k个符号
- 当k的数量增大时, 相应的语法分析器的规模急剧增大
 - K=2时, 程序设计语言的语法分析器的规模通常非常庞大
 - 当k=0、1时已经可以解决很多语法分析问题, 因此具有实践意义
 - 因此, 我们只考虑k≤1的情况



自底向上的语法分析



- 表格驱动
 - 虽然手工构造表格工作量大, 但表格可以自动生成
- 对于几乎所有的程序设计语言, 只要写出上下文无关文法, 就能够构造出识别该构造的LR语法分析器
- 最通用的无回溯移入-归约分析技术, 且和其它技术一样高效
- 可以尽早检测到错误
- LR(1) 可以识别所有确定性上下文无关文法
- 能分析的文法集合是LL(k)文法的超集



自底向上的语法分析



- 移入-归约语法分析器如何知道何时该移入、何时该归约呢?
 - LR语法分析器试图用一些状态来表明我们在移进归约语法分析过程中所处的位置, 从而做出移入-归约决定
- 项的意义
 - 指明在语法分析过程中的给定点上, 我们已经看到了一个产生式的哪些部分。或者说, 如果我们想用这个产生式进行归约, 还需要看到哪些文法符号
- 项的集合(项集)对应于一个状态



自底向上的语法分析



- LR(0)分析技术
- LR(0)项
 - 文法的一个产生式加上在其产生式体中某处的一个点
 - $A \rightarrow .XYZ$, $A \rightarrow X.YZ$, $A \rightarrow XY.Z$, $A \rightarrow XYZ.$
 - 注意: $A \rightarrow \epsilon$ 只对应一个项 $A \rightarrow .$
 - 直观含义
 - 项 $A \rightarrow \alpha.\beta$ 表示已经扫描/归约到了 α , 并期望接下来的输入中经过扫描/归约得到 β , 然后把 $\alpha\beta$ 归约到A
 - 如果 β 为空, 表示我们可以把 α 归约为A
 - 项也可以用一对整数表示
 - (i,j)表示第i条规则, 点位于右部第j个位置



自底向上的语法分析



- LR(0)分析技术
- 规范LR(0)项集族
 - 规范LR(0)项集族提供构建LR(0)自动机的基础
 - LR(0)自动机可用于做出语法分析决定
 - LR(0)自动机中每个状态代表了LR(0)项集族中的一个项集



自底向上的语法分析



- LR(0)分析技术
- 以项为基础构造自动机(DFA)
 - 构造以项为状态的自动机
 - 开始状态 $S' \rightarrow .S$
 - 转换
 - $A \rightarrow \alpha.B\beta$ 到 $A \rightarrow \alpha.X\beta$ 有一个 ϵ 转换
 - 从 $A \rightarrow \alpha.X\beta$ 到 $A \rightarrow \alphaX.\beta$ 有一个X转换
 - 接受状态: $A \rightarrow \alpha.$, 即点在最后的项



自底向上的语法分析



- LR(0) 分析技术
- 规范LR(0) 项集族的构造
- 三个概念
 - 增广文法
 - 项集闭包: CLOSURE
 - GOTO



自底向上的语法分析



- LR(0) 分析技术
- 规范LR(0) 项集族的构造
- 增广文法
 - G的增广文法G'是在G中增加新开始符号S', 并加入产生式S'→S而得到的
 - G'和G接受相同的语言, 且按照S'→S进行归约实际上就表示已经将输入符号串归约成为开始符号
 - 引入的目的是告诉语法分析器何时宣布接受输入符号串, 即用S'→S进行归约时, 表明分析结束



自底向上的语法分析



- LR(0) 分析技术
- 规范LR(0) 项集族的构造
 - CLOSURE(I): I的项集闭包
 - 对应于DFA化算法的ε-CLOSURE
 - GOTO(I,X): I的X后继
 - 对应于DFA化算法的MOVE(I,X)



自底向上的语法分析



- CLOSURE(I) 的构造算法
- 如果I是文法G的一个项集, 那么CLOSURE(I)就是根据下列规则从I构造得到的项集
 - 将I中的各个项加入到CLOSURE(I)中
 - 如果A → α.Bβ在CLOSURE(I)中, 那么对B的任意产生式B → γ, 将B → γ加到CLOSURE(I)中
 - 不断重复第二步, 直到收敛
- 第二步的意义
 - 项A → α.Bβ表示期望在接下来的输入中归约到B
 - 显然, 要归约到B, 首先要扫描归约到B的某个产生式的右部
 - 因此对每个产生式B → γ, 加入B → γ
 - 表示它期望能够扫描归约到γ



自底向上的语法分析



- 增广文法:
 - E' → E E → E+T | T T → T*F | F F → (E) | id
- 项集 I={ [E' → E] } 的闭包
 - [E' → E]在闭包中
 - [E → E+T], [E → T]在闭包中
 - [T → T*F], [T → F]在闭包中
 - [F → (E)], [F → id]在闭包中

```

SetOfItems CLOSURE(I) {
    J = I;
    repeat
        for (J中的每个项 A → α.Bβ)
            for (G 的每个产生式 B → γ)
                if (项 B → γ 不在 J 中)
                    将 B → γ 加入 J 中;
    until 在某一轮中没有新的项被加入到 J 中;
    return J;
}
  
```



自底向上的语法分析



- LR(0) 项集中的内核项和非内核项
 - 内核项: 初始项S' → S、以及所有点不在最左边的项
 - 非内核项: 除了S' → S之外、点在最左边的项
- 由于表可能很庞大, 实现算法时可以考虑只保存相应的非终结符号; 甚至可以只保存内核项, 而在要使用非内核项时调用CLOSURE函数重新计算



自底向上的语法分析

- GOTO函数
- I 是一个项集, X 是一个文法符号, $GOTO(I, X)$ 定义为 I 中所有形如的项 $[A \rightarrow \alpha \cdot X \beta]$ 所对应的项 $[A \rightarrow \alpha X \cdot \beta]$ 的集合的闭包
 - 根据项的历史-期望的含义, $GOTO(I, X)$ 表示读取输入中的 X 或者归约到一个 X 之后的情况
 - $GOTO(I, X)$ 定义了 LR(0) 自动机中状态 I 在 X 之上的转换
- 例如:
 - $I = \{[E' \rightarrow E.], [E \rightarrow E. + T]\}$
 - $GOTO(I, +)$ 计算如下
 - I 中只有一个项的点后面跟着 $+$, 对应的项为 $[E \rightarrow E. + T]$
 - $CLOSURE(\{[E \rightarrow E. + T]\}) = \{[E \rightarrow E. + T], [T \rightarrow T * F], [T \rightarrow (E)], [F \rightarrow (E)], [F \rightarrow id]\}$



自底向上的语法分析

- 规范 LR(0) 项集族的构造
- 为文法 G 构造 LR(0) 项集规范族 C 的关键步骤总结
 - LR(0) 项目: 形如 $A \rightarrow \alpha \cdot \beta$, 点 $(\cdot)$ 表示在该产生式“已经识别到”的位置
 - $closure(I)$: 若 $A \rightarrow \alpha \cdot B \beta \in I$ ($\cdot$ 在非终结符 B 前), 则把所有 $B \rightarrow \gamma$ 加入 I , 并重复直到不再能加入
 - $goto(I, X)$: 对集合 I 中所有 $A \rightarrow \alpha \cdot X \beta$, 把点向右移一位得 $A \rightarrow \alpha X \cdot \beta$, 然后对该新集合做 $closure$, 得到 $goto(I, X)$
 - 从初始 $I_0 = closure(\{E' \rightarrow \cdot E\})$ 出发, 反复计算 $goto$, 直到无新集合为止 $\rightarrow$ 得到项目集族 (states)

给出来一个 step by step 的例子?



自底向上的语法分析

- 规范 LR(0) 项集族的构造示例:
 - $goto(I_0, id)$: 由 $F \rightarrow \cdot id$ 得 $F \rightarrow id \cdot$ (这是一个完成项, 表示能规约 $F \rightarrow id$); $closure$ 不再添加新项
 - $goto(I_0, '(')$: 由 $F \rightarrow \cdot (E)$ 得 $F \rightarrow (\cdot E)$, 此时 $\cdot$ 前是非终结符 E , 因此需要把 $E \rightarrow \cdot E + T, E \rightarrow \cdot T$ 等产生式一并加入, 这会形成一个结构近似于 I_0 的子集 (但 $\cdot$ 在不同位置)
 - $goto(I_0, E)$: 由 $E' \rightarrow \cdot E$ 与 $E \rightarrow \cdot E + T$ 等项目得到 $E' \rightarrow E \cdot$ 与 $E \rightarrow E \cdot + T$, 这构成了一个状态 (可以记为 I_1), 表示“已识别一个完整的 E , 但也可看到 $+$ 接着更多的东西”
 - $goto(I_0, T), goto(I_0, F) \dots$



自底向上的语法分析

- 规范 LR(0) 项集族的构造
- 为文法 G 构造 LR(0) 项集规范族 C
 - 步骤一: 构造增广文法 G' : $S' \rightarrow S \dots$
 - 步骤二: 构造 $I_0 = CLOSURE(S' \rightarrow \cdot S)$, I_0 是 C 的初始项集
 - 步骤三: 对 C 中的每个项集 I_i 及每个文法符号 X_j , 将 $GOTO(I_i, X_j)$ 加入到 C 中
 - 重复步骤三, 直到没有新的项集可以加入

```
void items(G') {
    C = CLOSURE({S' → · S});
    repeat
        for (C 中的每个项集 I)
            for (每个文法符号 X)
                if (GOTO(I, X) 非空且不在 C 中)
                    将 GOTO(I, X) 加入 C 中;
    until 在某一轮中没有新的项集被加入 C 中;
}
```

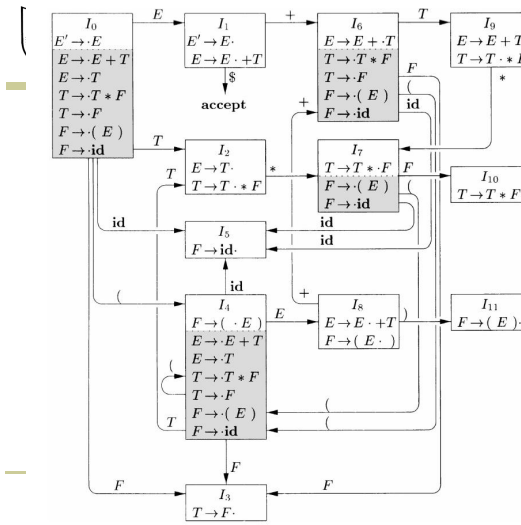


自底向上的语法分析

- 规范 LR(0) 项集族的构造示例:
 - 初始 $I_0 = closure(\{E' \rightarrow \cdot E\})$ 会包含下列项目:
 - $E' \rightarrow \cdot E$
 - $E \rightarrow \cdot E + T$
 - $E \rightarrow \cdot T$
 - $T \rightarrow \cdot T * F$
 - $T \rightarrow \cdot F$
 - $F \rightarrow \cdot (E)$
 - $F \rightarrow \cdot id$

说明: 因为 $\cdot$ 前面是 E (非终结符), 我们把 E 的产生式也加入; 在 $E \rightarrow \cdot T$ 中 $\cdot$ 前是 T , 我们继续把 T 的产生式加入; 同理把 F 的产生式加入, 直到 $closure$ 完成

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$



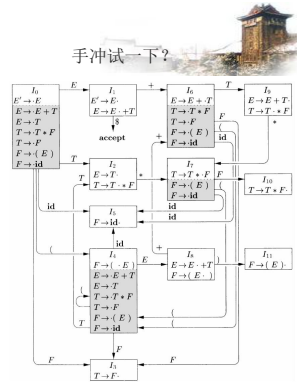
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

项集规范族和 GOTO 函数的例子



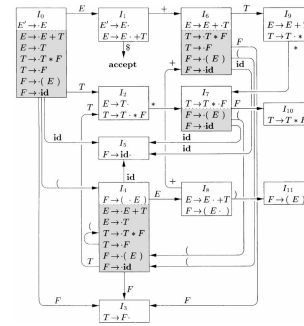
自底向上的语法分析

- 我们得到了如上的DFA，下一步，构建分
- 基本算法：
 - 对于每个状态 i 和输入符号 a ：
 - 移进 (Shift)
 - 如果某个项目 $[A \rightarrow \alpha \cdot a]$ 在状态 i
 - 那么 $\text{ACTION}[i, a] = "S_j"$ ，其中 $j = \text{goto}(i, a)$ 的
 - 规约 (Reduce)
 - 如果某个项目 $[A \rightarrow \alpha \cdot]$ 在状态 i (点在最右边)
 - ~~那么对所有 $a \in \text{FOLLOW}(A)$ ， $\text{ACTION}[i, a] = "R(A \rightarrow \alpha)"$~~
 - 接受 (Accept)
 - 如果有项目 $[E' \rightarrow E \cdot]$ 在状态 i ，并且输入符号是 $\$$ ，且 $\text{goto}(i, \$) = j$ ，则 $\text{GOTO}[i, \$] = j$
 - 那么 $\text{ACTION}[i, \$] = \text{ACC}$



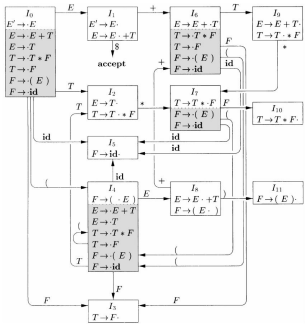
自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$ (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$ (6) $F \rightarrow \text{id}$



自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$ (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$ (6) $F \rightarrow \text{id}$

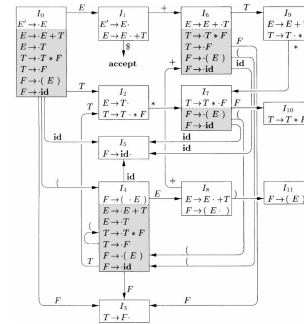


状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2				
3	r4	r4	r4	r4	r4				
4	s5	s4					8	2	3
5	r6	r6	r6	r6	r6				
6	s5	s4						9	3
7	s5	s4							10
8				s11	s6				
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3				
11	r5	r5	r5	r5	r5				



自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$ (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$ (6) $F \rightarrow \text{id}$

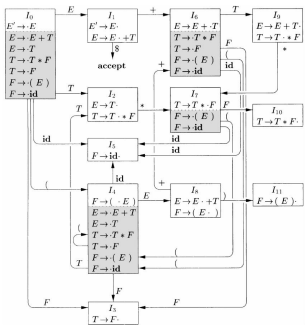


状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2				
3	r4	r4	r4	r4	r4				
4	s5	s4					8	2	3
5	r6	r6	r6	r6	r6				
6	s5	s4						9	3
7	s5	s4							10
8				s11	s6				
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3				
11	r5	r5	r5	r5	r5				



自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$ (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$ (6) $F \rightarrow \text{id}$



在上表中出现 Shift/Reduce 冲突的格子为：
 State 2，在符号 * 上：表格格子是 s7/r2: $E \rightarrow T$ (冲突)。
 State 2 的 LR(0) 项集为：
 $E \rightarrow T \cdot$ (可以用 $E \rightarrow T$ 归约)
 $T \rightarrow T \cdot * F$ (看到 * 时可以 shift，进入乘法右侧)
 因为 LR(0) 在没有 lookahead 的情况下会在所有终结符上进行归约，所以在 * 上就同时出现了 shift (为处理乘法) 与 reduce (把当前 T 归约为 E) 两种动作 —— 无法在 LR(0) 中决定，产生冲突。

State 9，在符号 * 上：表格格子是 s7/r1: $E \rightarrow E + T$ (冲突)。
 State 9 的 LR(0) 项集为：
 $E \rightarrow E + T \cdot$ (可以用 $E \rightarrow E + T$ 归约)
 $T \rightarrow T \cdot * F$ (看到 * 时可以 shift)
 同理，既可以把 $E + T$ 归约为 E (r1)，又可能因为接下来看到 * 而需要做乘法结合 (shift)，LR(0) 无法区分，发生冲突。

状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2				
3	r4	r4	r4	r4	r4				
4	s5	s4					8	2	3
5	r6	r6	r6	r6	r6				
6	s5	s4						9	3
7	s5	s4							10
8				s11	s6				
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3				
11	r5	r5	r5	r5	r5				



自底向上的语法分析

- 假设文法符号串 γ 使 LR(0) 自动机从开始状态运行到状态 (项集) j
 - 如果 j 中有一个形如 $A \rightarrow \alpha$ 的项，那么
 - 在 γ 之后添加一些终结符号可以得到一个最右句型
 - α 是 γ 的后缀，且 $A \rightarrow \alpha$ 是这个句型的句柄
 - 表示可能找到了当前最右句型的句柄
 - 如果 j 中存在一个项 $B \rightarrow \alpha.X\beta$ ，那么
 - 在 γ 之后添加 $X\beta$ ，然后再添加一个终结符号串可得到一个最右句型
 - 在这个句型中 $B \rightarrow \alpha.X\beta$ 是句柄
 - 此时表示还没有找到句柄，需要移入



自底向上的语法分析



- LR(0)自动机的使用
 - 移入-归约时，LR(0)自动机被用于识别句型
 - 已经归约/移入得到的文法符号序列对应于LR(0)自动机的一条路径
 - 如果路径到达接受状态，表明栈上端的某个符号串可能是句柄
- 不需要每次用归约/移入得到的串来运行LR(0)自动机
 - 路径被放到栈中；且文法符号可以省略，因为由LR(0)状态可以确定相应文法符号
 - 在移入后，根据原来的栈顶状态即可知道新的状态
 - 在归约时，根据归约产生式的右部长度的右部长度弹出相应状态，仍然可以根据此时的栈顶状态计算得到新状态



自底向上的语法分析



- 一个关键的问题？！
- LR(0) 在做“规约”决策时不看任何前瞻符号
 - 也就是说一旦某个项目集里出现了完成项 $A \rightarrow \alpha \cdot$ ，LR(0) 会在所有可能的输入符号上把它当作可规约动作（除非该状态上还有移进动作，产生移进-规约冲突）
- 对于这个表达式文法，许多状态会同时包含：
 - 某个完成项（例如 $F \rightarrow id \cdot$ 或 $T \rightarrow F \cdot$ 等），以及
 - 另一个项目如 $T \rightarrow T \cdot * F$ 或 $E \rightarrow E \cdot + T$ （点在非终结符或终结符前）
- 因此，当分析器遇到某些输入（例如 $id + id * id$ 的中间位置），会产生移进-规约冲突或规约-规约冲突

LR(0) 无法凭借其有限状态做出正确、唯一的决策



自底向上的语法分析



- 回顾：
 - First函数：由某个非终结符能够推导出来的句子的句首符号的集合，就是这个非终结符的First集合
 - Follow函数：对于某个非终结符，紧跟在其后面的终结符的集合就是这个非终结符的Follow集



自底向上的语法分析



- 一个关键的问题？！
- LR(0) 在做“规约”决策时不看任何前瞻符号
 - 也就是说一旦某个项目集里出现了完成项 $A \rightarrow \alpha \cdot$ ，LR(0) 会在所有可能的输入符号上把它当作可规约动作（除非该状态上还有移进动作，产生移进-规约冲突）



自底向上的语法分析



- 讨论：LR(0) 的局限性
 - 有些文法会产生 移进-规约冲突 或 规约-规约冲突
 - LR(0) 过于严格，能处理的文法较少
 - 根本原因：
 - LR(0) 只看项目中的“点位置”，并不知道后面真正可能出现的符号
 - 因此在“能规约”和“能移进”的情形下，它无法做出唯一选择
- 改进方案是什么？
 - 更强的算法（SLR, LR(1), LALR）就是通过前瞻或 FOLLOW 来减少冲突



自底向上的语法分析



- SLR: Simple LR 算法的核心
 - 基本思想：
 - 在 LR(0) 项目集基础上，加上 FOLLOW 集来控制规约动作
 - 规约 $A \rightarrow \alpha$ 只在 输入符号 $\in \text{FOLLOW}(A)$ 时才执行
 - 优点：
 - 消除了大部分 LR(0) 的冲突
 - 构造过程和 LR(0) 类似，只需在生成 ACTION 表时加上 FOLLOW 集限制
 - 缺点：
 - 仍然可能存在不足（FOLLOW 集过大，可能引入不必要的冲突）



自底向上的语法分析

- SLR: Simple LR 算法的核心
- Follow集的计算:



自底向上的语法分析

- SLR: Simple LR 算法的核心
 - 如果不仅考虑First, 而且把Follow集也考虑在内, 就可以一定程度上解决reduce-shift conflict. 还是以上语法:
 - $F \rightarrow Y \cdot + B$
 - $F \rightarrow Y \cdot$
 - 如果 $FOLLOW(F) = \{a, b\}$, 那么当我们遇到a与b时, 就可以选择归约F; 当我们遇到+时, 就可以选择移进操作



自底向上的语法分析

- 输入: 一个增广文法G'
- 输出: G'的SLR语法分析表函数ACTION和GOTO
- 方法:
 - 1) 构造G' LR(0)项集规范族 $C = \{I_0, I_1, \dots, I_n\}$
 - 2) 对于状态i中的项:
 - $[A \rightarrow \alpha \cdot a \beta]$ 且 $GOTO[I_i, a] = I_j$, 那么 $ACTION[i, a] = \text{移入 } j$
 - $[A \rightarrow \alpha \cdot]$, 那么对于 $FOLLOW(A)$ 中的所有a, $ACTION[i, a] = \text{归约 } A \rightarrow \alpha$
 - $[S' \rightarrow S \cdot]$, 那么 $ACTION[i, \$] = \text{接受}$
 - 3) 状态i对于非终结符号A的转换规则:
 - 如果 $GOTO[I_i, A] = I_j$, 那么 $GOTO[i, A] = j$
 - 4) 规则2)和3)没有定义的所有条目设置为“报错”



自底向上的语法分析

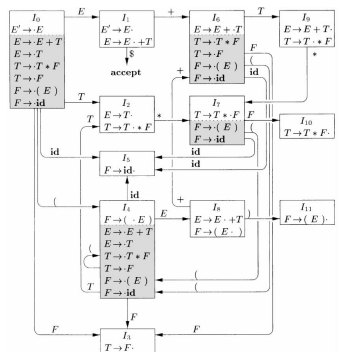
- SLR: Simple LR 算法的核心
 - (0) $E' \rightarrow E$
 - (1) $E \rightarrow E + T$
 - (2) $E \rightarrow T$
 - (3) $T \rightarrow T * F$
 - (4) $T \rightarrow F$
 - (5) $F \rightarrow (E)$
 - (6) $F \rightarrow id$
- FOLLOW 集结果:
 - $FOLLOW(E) = \{ \$,), + \}$
 - 说明: 来自 $E' \rightarrow E$ 得到 $\$$; $F \rightarrow (E)$ 得到 $)$; $E \rightarrow E + T$ 在 $+$ 后面有 T , 与 $FOLLOW(E)$ 无关; 但在 $E \rightarrow E + T$ 中, E 后面直接是 $+$, 因此 $+$ $\in FOLLOW(E)$.
 - $FOLLOW(T) = \{ \$,), + \}$
 - 说明: $E \rightarrow E + T$ 给 $FOLLOW(T)$ 包含 $FOLLOW(E)$; 且 T 在 $E \rightarrow E + T$ 末尾, $FOLLOW(T) \supseteq FOLLOW(E)$; 综合为 $\{ \$,), + \}$.
 - $FOLLOW(F) = \{ \$,), +, * \}$
 - 说明: $T \rightarrow T * F$ 给 $FOLLOW(F)$ (F 前面是 $*$) 但 F 在式尾, $FOLLOW(F) \supseteq FOLLOW(T)$; 同时在 $T \rightarrow T * F$ 中, F 后无符号, 故 $FOLLOW(F) \supseteq FOLLOW(T)$; 最终 $\{ \$,), +, * \}$.
- 在某状态出现 $[E \rightarrow T \cdot]$
 - 在 LR(0): 会在所有终结符上放 "reduce $E \rightarrow T$ "
 - 所以在 $*$ 上冲突 (因为 $[T \rightarrow T * F]$ 允许移进)
 - 在 SLR: 只在 $FOLLOW(E) = \{ +,), \$ \}$ 上放规约
 - 在 $*$ 上不会规约 $\rightarrow$ 冲突消除了

$FOLLOW(E) = \{ +,), \$ \}$
 $FOLLOW(T) = \{ +, *,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$



自底向上的语法分析

- SLR: Simple LR 算法的核心

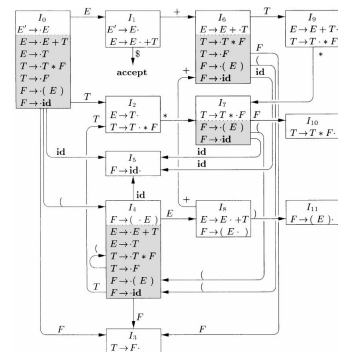


	(0) $E' \rightarrow E$										
	(1) $E \rightarrow E + T$										
	(2) $E \rightarrow T$										
	(3) $T \rightarrow T * F$										
	(4) $T \rightarrow F$										
	(5) $F \rightarrow (E)$										
	(6) $F \rightarrow id$										
	$FOLLOW(E) = \{ +,), \$ \}$										
	$FOLLOW(T) = \{ +, *,), \$ \}$										
	$FOLLOW(F) = \{ +, *,), \$ \}$										
状态	id	(	)	+	*	\$	E	T	F		
0	s5	s4					1	2	3		
1				s6	acc						
2	r2	r2	r2	r2	s7/r2	r2					
3	r4	r4	r4	r4	r4	r4					
4	s5	s4					8	2	3		
5	r6	r6	r6	r6	r6	r6					
6	s5	s4						9	3		
7	s5	s4							10		
8				s11	s6						
9	r1	r1	r1	r1	s7/r1	r1					
10	r3	r3	r3	r3	r3	r3					
11	r5	r5	r5	r5	r5	r5					



自底向上的语法分析

- SLR: Simple LR 算法的核心



	(0) $E' \rightarrow E$										
	(1) $E \rightarrow E + T$										
	(2) $E \rightarrow T$										
	(3) $T \rightarrow T * F$										
	(4) $T \rightarrow F$										
	(5) $F \rightarrow (E)$										
	(6) $F \rightarrow id$										
	$FOLLOW(E) = \{ +,), \$ \}$										
	$FOLLOW(T) = \{ +, *,), \$ \}$										
	$FOLLOW(F) = \{ +, *,), \$ \}$										
状态	id	(	)	+	*	\$	E	T	F		
0	s5	s4					1	2	3		
1				s6	acc						
2				r2	r2	s7	r2				
3				r4	r4	r4	r4				
4	s5	s4					8	2	3		
5				r6	r6	r6	r6				
6	s5	s4						9	3		
7	s5	s4							10		
8				s11	s6						
9				r1	r1	s7	r1				
10				r3	r3	r3	r3				
11				r5	r5	r5	r5				





- 根据上一算法构造的语法分析表，若表中各位置没有多个条目，则称为文法G的SLR(1)分析表。使用该分析表的分析器，称为G的SLR(1)语法分析器。G称为SLR(1)文法
- 若表中某位置存在多个条目（多个可选的动作，冲突），则该文法是非SLR(1)文法

总结：SLR(1) = 在 LR(0) 基础上 + FOLLOW 过滤

→ 简单高效，但能力有限；LR(1) 更强，但代价是状态数暴增。



- SLR语法分析器的弱点
 - SLR不能完全解决reduce-shift conflict.
 - 同样还是上面的语法：
 - $F \rightarrow Y \cdot + B$
 - $F \rightarrow Y \cdot$
 - 如果 $FOLLOW(F) = \{a, b, +\}$ ，那当我们遇到 + 符号时，就无法确定到底是选择移进操作得到 $F \rightarrow Y + \cdot B$ ，还是归约F。
 - SLR不能完全解决reduce-shift conflict. 这就是为什么我们要选择LR(1) / LALR(1)了